



دانشکده‌گان علوم و فناوری‌های میان‌رشته‌ای

دانشکده سامانه‌های هوشمند

گروه علوم و فناوری داده

درس کاربرد سیستم‌های هوشمند در پزشکی

استاد: دکتر مهدی تیموری

گردآورنده: مهدیس باقری دورفرد

INDEX

Imports	3
Load data	3
MATLAB Functions.....	4
Task One: DNF Learning	5
1. Learn Single Rule	5
2. Learn a Full DNF.....	8
3. Predict using the Learned DNF	10
4. Turn the DNF model into a readable string	11
5. Main Experiment: Train Evaluation Loop.	11
6. Plot Learning Curve	12
Task Two: Noisy DNF Learning.....	13
1. Learn Single Rule	13
2. Learn a Full DNF.....	15
3. Predict using the Learned DNF	16
4. Turn the DNF model into a readable string	16
5. Main Experiment: Epsilon sweep.	17
6. Plot Error vs Epsilon curve.....	18
Analysis and Results	19
Conclusion	21

Imports

```
from scipy.io import loadmat
```

برای خواندن فایل‌های MATLAB

```
import numpy as np
```

کتابخانه اصلی محاسبات عددی است و در کد برای کار با آرایه‌ها، محاسبه خطا و عملیات منطقی استفاده می‌شود.

```
import matplotlib.pyplot as plt
```

برای رسم و نمایش نمودارها مثل learning curve و Error vs Epsilon Curve استفاده می‌شود.

Load data

```
DNF_Data = loadmat('DNFData.mat')
```

فایل DNFData.mat که یک فایل MATLAB است خوانده می‌شود و محتوای آن به صورت یک دیکشنری در متغیر DNF_Data ذخیره می‌شود.

```
data = DNF_Data['Data']
```

از داخل دیکشنری، متغیر Data استخراج می‌شود که یک ماتریس شامل کل dataset است.

```
x = data[:, :-1]  
y = data[:, -1]
```

تمام ستون‌ها به جز ستون آخر به عنوان ویژگی‌ها (features) انتخاب می‌شوند. ستون آخر ماتریس به عنوان برچسب کلاس (label) استخراج می‌شود که نشان می‌دهد هر نمونه مثبت یا منفی است.

سپس همین مراحل برای لود کردن DNF نویزی تکراری می‌شود.

در توضیحات تمرین ذکر شده بود که در صورت استفاده از MATLAB، دستورات `intersect`، `setdiff`، `any`، `all` و `plot` می‌توانند در پیاده‌سازی الگوریتم مفید باشند. در پیاده‌سازی این تمرین از زبان Python و کتابخانه‌های NumPy و Matplotlib استفاده شد و برای هر یک از این دستورات، معادل مناسب در پایتون به کار گرفته شد.

دستور `all` در MATLAB برای بررسی برقرار بودن یک شرط برای تمام عناصر یک مجموعه استفاده می‌شود. در پیاده‌سازی پایتونی از تابع `np.all` استفاده شد.

دستور `any` در MATLAB برای بررسی این است که آیا حداقل یکی از عناصر یک مجموعه شرط موردنظر را برآورده می‌کند یا خیر. در کد پایتون نیز از معادل مستقیم آن یعنی `np.any` استفاده شده است، برای مثال جهت بررسی خالی نبودن یک `rule` در مدل.

دستور `setdiff` در MATLAB برای محاسبه اختلاف دو مجموعه استفاده می‌شود. در این پیاده‌سازی از معادل آن در NumPy یعنی `np.setdiff1d` یا فیلترهای منطقی استفاده شده است.

دستور `intersect` در MATLAB برای یافتن اشتراک دو مجموعه به کار می‌رود. در پیاده‌سازی پایتون این عمل معمولاً با استفاده از فیلترهای منطقی و انتخاب برداری (boolean indexing) انجام شده است تا نمونه‌هایی که هم‌زمان چند شرط را ارضا می‌کنند انتخاب شوند. بنابراین اگرچه تابع `np.intersect1d` به صورت مستقیم استفاده نشده است، اما همان مفهوم در بخش‌های مختلف الگوریتم پیاده‌سازی شده است.

در نهایت برای رسم نمودارها که در MATLAB با دستور `plot` انجام می‌شود، در این تمرین از کتابخانه Matplotlib و تابع `plt.plot` استفاده شده است. این دستور برای رسم منحنی یادگیری و همچنین نمودار تغییرات خطای اعتبارسنجی نسبت به مقادیر مختلف پارامتر ϵ به کار رفته است.

تمام دستورات پیشنهادی MATLAB در این تمرین در قالب معادل‌های مناسب در Python پیاده‌سازی شده‌اند و همان عملکرد موردنظر را در فرآیند یادگیری و ارزیابی مدل فراهم کرده‌اند.

تابع Learn Single Rule (Conjunction) 🚦

این تابع یک قانون از نوع conjunction یاد می‌گیرد؛ یعنی قانونی از نوع: $f1 \text{ AND } f3 \text{ AND } f7$ که هدفش این است:

- مثال‌های مثبت (Positive) را پوشش دهد
- مثال‌های منفی (Negative) را حذف کند

```
def learn_single_rule(P_indices, X, y):
```

ورودی تابع: اندیس نمونه‌های مثبت (P_indices) و ماتریس داده‌ها (Feature Matrix) و بردار تارگت‌ها

ایده اصلی الگوریتم:

۱. ابتدا یک قانون خالی داریم
۲. کم‌کم ویژگی‌ها را به قانون اضافه می‌کنیم
۳. تا وقتی ادامه می‌دهیم که همه مثال‌های منفی حذف شوند

```
num_features = X.shape[1]
```

```
x.shape
✓ 0.0s
(2000, 100)
```

متغیر num_features، تعداد ویژگی‌ها (ستون‌های ماتریس X) است.

```
r_features = np.zeros(num_features, dtype=bool)
```

در r_features یک بردار بولی می‌سازیم که نشان می‌دهد کدام ویژگی‌ها در قانون هستند (هر کدام که True باشند). مثلاً اگر تعداد فیچر‌ها ۴ باشد، در ابتدا داریم: [False False False False] یعنی هنوز هیچ ویژگی در قانون نیست.

```
N_idx = np.where(y == 0)[0]
```

اندیس نمونه‌های منفی استخراج می‌شود و در N_idx ذخیره می‌شود.

```
current_N = list(N_indices)
```

لیست current_N، شامل مثال‌های منفی است که هنوز توسط قانون حذف نشده‌اند. هدف الگوریتم این است که current_N خالی شود.

```
while len(current_N) > 0:
```

حلقه‌ی اصلی الگوریتم:

تا وقتی هنوز مثال منفی باقی مانده، قانون را بهتر می‌کنیم.

```
if np.all(r_features):
    return None
```

اگر همه ویژگی‌ها استفاده شده باشند (همه مقدار True داشته باشند) الگوریتم شکست خورده. چون دیگر ویژگی جدیدی برای اضافه کردن نداریم ولی لیست current_N خالی نشده است.

```
best_f = None
best_score = -np.inf
```

best_f اندیس بهترین ویژگی و best_v مقدار هیوریستیک آن ویژگی است که در اینجا initialize شده اند.

```
for f in range(num_features):
```

برای هر ویژگی بررسی می کنیم که آیا اضافه کردن آن به قانون مفید است؟

```
if r_features[f]:
    continue
```

اگر ویژگی انتخابی قبلاً در قانون آمده باشد (مقدار آن در r_features True باشد) از آن رد می شویم.

```
temp_r = r_features.copy()
temp_r[f] = True
```

در این بخش کد، ابتدا r_features در temp_r کپی میشود. سپس، ویژگی انتخاب شده یا همان f مقدار True میگیرد (به صورت موقت)

```
n_pos = np.sum(np.all(X[P_indices][:, temp_r] == 1, axis=1))
```

این قسمت کد، بررسی میکند که چند نمونه مثبت توسط قانون جدید پوشش داده می شوند.

```
n_neg = np.sum(np.all(X[current_N][:, temp_r] == 1, axis=1))
```

این قسمت بررسی میکند چند نمونه منفی هنوز با این ویژگی سازگار هستند.

```
Score = n_pos / max(n_neg, 0.001)
```

$$v = \frac{n_{pos}}{n_{neg}}$$

این بخش هیوریستیک را محاسبه میکند. یعنی:

- می خواهیم مثبت ها زیاد شوند
- منفی ها کم شوند

برای جلوگیری از تقسیم بر صفر، از ماکسیموم n_neg و 0.001 استفاده میشود.

```
if score > best_score:
    best_score = score
    best_f = f
```

در این حلقه ی for، ویژگی ای انتخاب می شود که بیشترین مقدار heuristic را داشته باشد.

```
if best_f is None:
    return None
```

اگر `best_f` همچنان `None` مانده باشد، یعنی هیچ ویژگی مفیدی پیدا نشده. پس الگوریتم شکست می خورد.

```
r_features[best_f] = True
```

اگر `best_f` پیدا شده باشد، مقدار آن در `r_features`، `True` میشود.

```
current_N = [idx for idx in current_N if X[idx, best_f] == 1]
```

در این بخش کد، نمونه های منفی که $f = 0$ دارند حذف میشوند.

سپس الگوریتم به ابتدای حلقه ی `while` برمیگردد.

🚩 تابع Learn a Full DNF (Disjunction of Conjunctions)

این تابع یک فرمول DNF را با استفاده از الگوریتم Sequential Covering یاد می‌گیرد. ابتدا همه نمونه‌های مثبت و منفی از داده آموزشی جدا می‌شوند و سپس در هر مرحله تابع `learn_single_rule` یک `conjunction rule` می‌سازد که تعدادی از `positive`ها را پوشش داده و همزمان `negative`ها را رد کند. پس از یادگیری هر `rule`، نمونه‌های مثبت پوشش داده شده حذف می‌شوند و `rule` به مدل نهایی DNF اضافه می‌شود. این روند تا زمانی ادامه پیدا می‌کند که تمام `positive`ها پوشش داده شوند؛ در غیر این صورت، اگر `rule` معتبری پیدا نشود، تابع مقدار `None` برمی‌گرداند.

```
def learn_DNF(X_train, y_train):
```

ورودی‌های این تابع: ماتریس ویژگی‌ها و بردار برچسب‌ها

```
P = list(np.where(y_train == 1)[0])
N_all = list(np.where(y_train == 0)[0])
```

در این بخش، نمونه‌های مثبت و منفی پیدا میشوند. تابع `np.where` اندیس‌هایی را برمی‌گرداند که شرط داخل پرانتز برایشان برقرار باشد.

```
hypo_rules = []
```

این لیست قرار است `rule`های DNF را نگه دارد. مثلاً در نهایت ممکن است شود: `[r1, r2, r3]` که یعنی `r1 OR r2 OR r3`

```
while len(P) > 0:
```

تا وقتی `positive`هایی داریم که هنوز پوشش داده نشده‌اند، الگوریتم ادامه دارد

```
r = learn_single_rule(P, N_all, X_train)
```

تابع `learn_single_rule` که توضیح داده شد، `call` میشود و قانون `r` که یک `conjunction` است، تولید میشود.

```
if r is None:
    return None
hypo_rules.append(r)
```

اگر هیچ قانونی پیدا نشود، الگوریتم اعلام شکست میکند. چون دیگر نمی‌توان `positive`ها را از `negative`ها جدا کند. در غیر این صورت قانون `r` به `hypo_rules` افزوده میشود. اگر قبلاً داشتیم `r1` حالا میشود `r1 OR r2`

```
covered_in_P = [idx for idx in P if np.all(X_train[idx, r] == 1)]
```

حال باید اندیس سِمپل‌های `positive`های پوشش داده شده را پیدا کنیم (تا بعداً از لیست `P` حذفشان کنیم) در واقع باید برای هر `positive` بررسی شود که آیا `rule` روی آن برقرار است یا خیر. مثال:

```
rule: feature0 AND feature2
sample : [1,0,1]
because f0 = 1 f2 = 1
the sample is covered.
```



```
if len(covered_in_P) == 0:  
    return None
```

اگر rule حتی یک positive را هم پوشش ندهد، یعنی rule بی فایده است. پس الگوریتم شکست می خورد.

```
P = [idx for idx in P if idx not in covered_in_P]
```

positive هایی که rule پوشش داده را حذف می کنیم. چون دیگر لازم نیست برای آنها rule جدید بسازیم.

```
return hypo_rules
```

خروجی تابع: لیست rule ها.

تایع Predict using the Learned DNF

این تایع مرحله پیش‌بینی مدل DNF را انجام می‌دهد؛ یعنی بعد از اینکه rule ها در مرحله آموزش یاد گرفته شدند، تایع بررسی می‌کند نمونه‌های جدیدی که به مدل داده می‌شوند positive هستند یا negative. برای هر نمونه، تمام rule های موجود در فرمول DNF بررسی می‌شوند و اگر حتی یکی از rule ها برقرار باشد، آن نمونه positive (برجسب ۱) در نظر گرفته می‌شود؛ در غیر این صورت نمونه negative (برجسب ۰) پیش‌بینی می‌شود.

```
def predict_DNF(X, rules):
```

ورودی های این تایع شامل سمپل های جدید (که تایع در فرایند آموزش آن ها را ندیده) و قانون هایی هست که یاد گرفته است.

```
    if rules is None:
        return np.zeros(len(X), dtype=int)
```

اگر هیچ rule ای وجود نداشته باشد، همه نمونه ها صفر پیش‌بینی میشوند. چون مدل چیزی یاد نگرفته.

```
    final_preds = np.zeros(len(X), dtype=bool)
```

در این بخش کد، مقداردهی اولیه prediction ها انجام میشود. یعنی در ابتدا داریم: [False, False, False, ...] یعنی در ابتدا فرض میکنیم هیچ نمونه ای مثبت نیست. بعد کمکم rule ها را اعمال میکنیم.

```
    for r in rules:
```

حلقه ی for روی rule ها انجام میشود.

مثلاً: $r = [\text{True}, \text{False}, \text{True}, \text{False}]$ یعنی feature0 AND feature2

```
        condition = np.all(X[:, r] == 1, axis=1)
```

این خط بررسی می‌کند که کدام نمونه‌ها تمام feature های موجود در rule را دارند. ابتدا $X[:, r]$ از ماتریس داده X همه سطرها را نگه می‌دارد ولی فقط ستون‌هایی را انتخاب می‌کند که در rule مقدار True دارند؛ یعنی فقط feature هایی که جزو قانون هستند. سپس $\text{np.all}(\dots, \text{axis}=1)$ روی هر سطر جداگانه می‌کند آیا مقدار آن feature ها برابر ۱ است یا نه، و یک ماتریس True/False می‌سازد. بعد $\text{np.all}(\dots, \text{axis}=1)$ روی هر سطر جداگانه بررسی می‌کند که آیا همه مقدارهای آن سطر True هستند یا نه. اگر همه True باشند یعنی آن نمونه تمام feature های rule را دارد و قانون برایش برقرار است؛ در نتیجه خروجی آن سطر True می‌شود، وگرنه False. بنابراین متغیر condition در نهایت یک آرایه بولی است که مشخص می‌کند هر نمونه توسط این rule پوشش داده می‌شود یا نه.

```
        final_preds = np.logical_or(final_preds, condition)
```

این خط دقیقاً مفهوم DNF را پیاده می‌کند. یعنی: $r1 \vee r2 \vee r3$. اگر حتی یکی از rule ها برقرار باشد، آن نمونه positive است.

```
    return final_preds.astype(int)
```

تبدیل bool به int (تبدیل True, False به ۱ و ۰).

Turn the DNF model into a readable string تابع

این تابع مدل DNF یادگرفته شده را از حالت داخلی و ماشینی به یک عبارت منطقی قابل خواندن برای انسان تبدیل می کند. اگر مدل خالی باشد یا هیچ rule ای وجود نداشته باشد، مقدار False برگردانده می شود؛ یعنی مدل هیچ شرطی برای positive بودن پیدا نکرده است. سپس تابع روی تمام rule ها حلقه می زند و با استفاده از `np.where(r)[0]` اندیس feature هایی را که در آن rule فعال هستند پیدا می کند. بعد هر feature را به شکل نمادهای منطقی مثل `x1`، `x2` و ... تبدیل می کند.

برای هر rule، feature ها با عملگر AND یعنی $\wedge$ به هم متصل می شوند و داخل پرانتز قرار می گیرند؛ مثلاً اگر یک rule شامل feature های ۱ و ۷ باشد، به صورت $(x1 \wedge x7)$ نمایش داده می شود. در نهایت تمام rule ها با عملگر OR یعنی $\vee$ به هم وصل می شوند تا فرمول کامل DNF ساخته شود؛ مثلاً $(x1 \wedge x7) \vee (x3)$

Main Experiment: Train Evaluation Loop

این بخش در واقع یک آزمایش تجربی برای بررسی عملکرد الگوریتم یادگیری DNF است. فرض اصلی این است که دیتاست بدون نویز (noiseless) است؛ یعنی برجسب ها کاملاً با یک فرمول DNF واقعی سازگار هستند و هیچ نمونه اشتباه یا متناقضی وجود ندارد. بنابراین از نظر تئوری، اگر داده آموزشی کافی داشته باشیم، الگوریتم باید بتواند فرمول درست را پیدا کند و خطا به صفر برسد.

```
train_sizes = np.arange(20, 1001, 20)
```

یعنی قرار است مدل را با اندازه های آموزشی ۲۰، ۴۰، ۶۰، ... تا ۱۰۰۰ نمونه آموزش دهیم. هدف این است که ببینیم با افزایش تعداد داده های آموزشی، خطای مدل چگونه تغییر می کند

```
errors = []
zero_error_size = None
final_dnf_model = None
```

آرایه errors برای ذخیره خطای هر مرحله استفاده می شود. متغیر zero_error_size اندازه اولین دیتاست آموزشی ای را نگه می دارد که در آن خطا صفر شده است و final_dnf_model نیز مدل DNF متناظر با آن نقطه را ذخیره می کند.

```
for t in train_sizes:
```

در هر بار اجرا حلقه، فقط `t` نمونه اول به عنوان داده آموزشی انتخاب می شوند

```
X_train, y_train = x[:t], y[:t]
X_val, y_val = x[t:], y[t:]
```

و بقیه داده ها برای validation استفاده می شوند. یعنی مدل روی بخش اول آموزش می بیند و روی داده هایی که ندیده ارزیابی می شود.

```
rules = learn_DNF(X_train, y_train)
if rules is None:
    errors.append(1.0)
```

اگر تابع `learn_DNF` نتواند یک مدل معتبر پیدا کند، مقدار `None` برمی گرداند. در این حالت کد فرض می کند مدل کاملاً شکست خورده و خطای ۱۰۰٪ ثبت می کند

```
else:
    y_pred = predict_DNF(X_val, rules)
```

اما اگر یادگیری موفق باشد، مدل روی داده های `validation` پیش بینی انجام می دهد و پیش بینی در `y_pred` ذخیره میشود.

```
err = np.mean(y_pred != y_val)
errors.append(err)
```

و بعد نرخ خطا محاسبه می شود.

عبارت `y_pred != y_val` یک آرایه بولی تولید می کند که در آن:

- `True` یعنی پیش بینی اشتباه
- `False` یعنی پیش بینی درست

تابع `np.mean` میانگین این مقادیر را می گیرد. چون در `NumPy` مقدار `True=1` و `False=0` است، میانگین در واقع درصد خطا می شود.

```
if err == 0 and zero_error_size is None:
    zero_error_size = t
    final_dnf_model = rules
```

این شرط بررسی می کند:

- آیا خطای `validation` صفر شده و برای اولین بار صفر شده؟

اگر بله، اندازه دیتاست آموزشی `t` فعلی ذخیره می شود و و مدل `DNF` متناظر نیز نگه داشته می شود.

Plot Learning Curve 📊

در این بخش، نمودار منحنی یادگیری (`Learning Curve`) مدل `DNF` را رسم می کند تا نشان دهد با افزایش تعداد داده های آموزشی، نرخ خطای مدل چگونه تغییر می کند.

تابع Learn Single Rule (Conjunction)

این تابع نیز مانند Learn Single Rule تسک قبلی یک conjunction rule تکی را برای الگوریتم DNF یاد می‌گیرد. کار را با یک rule خالی شروع می‌کند و در هر مرحله ویژگی‌ای را انتخاب می‌کند که بهترین نسبت پوشش positive ها به negative ها را داشته باشد.

تفاوت‌های learn_single_rule_noisy نسبت به نسخه ساده:

۱. اضافه شدن پارامتر epsilon

```
def learn_single_rule_noisy(X, y, P_idx, epsilon):
```

کاربرد:

- تعیین میزان خطای قابل قبول روی نمونه‌های منفی
- الگوریتم دیگر مجبور نیست همه negative ها را حذف کند.

در نسخه ساده هیچ tolerance برای noise وجود نداشت.

۲. تغییر معیار توقف الگوریتم

در نسخه ساده معیار توقف `while len(current_N) > 0` بود.

```
nn = epsilon * len(N_idx)
while len(N_idx) > nn:
```

در اینجا، الگوریتم زمانی متوقف می‌شود که تعداد negative های باقی‌مانده کمتر از آستانه نویز شود.

یعنی:

- اجازه می‌دهیم بخشی از negative ها باقی بمانند
- چون فرض می‌کنیم داده نویزی است و حذف کامل ممکن است overfitting ایجاد کند.

۳. حذف شرط شکست کامل

در نسخه ساده، اگر همه feature ها استفاده می‌شدند ولی هنوز negative باقی بود، یادگیری شکست می‌خورد.

```
if np.all(r_features) → return None
```

در نسخه نویزی این بخش حذف شده است. دلیل:

- در داده noisy ممکن است حذف کامل negativeها غیرممکن باشد.
- بنابراین الگوریتم tolerantتر طراحی شده است.

۴. اضافه شدن مکانیزم عدم پیشرفت

```
if (not made_progress) or (best_f is None):
    break
```

در نسخه نویزی، اگر feature مناسبی پیدا نشود، الگوریتم متوقف می‌شود. در نسخه ساده: شکست کامل رخ می‌داد، ولی در نسخه noisy الگوریتم rule ناقص را قبول می‌کند.

۵. تغییر مفهوم خروجی rule

در نسخه ساده:

- rule باید تمام negativeها را حذف کند.

در نسخه noisy:

- rule فقط باید بیشتر negativeها را حذف کند.
- باقی ماندن بخشی از negativeها مجاز است.

🚩 تابع Learn a Full DNF (Disjunction of Conjunctions)

این تابع با استفاده از روش Sequential Covering یک مدل DNF را در حالت داده‌های نویزی یاد می‌گیرد. در هر تکرار یک clause با استفاده از learn_single_rule_noisy ساخته می‌شود که تعدادی از نمونه‌های مثبت را پوشش می‌دهد و سپس آن positive ها از مجموعه حذف می‌شوند. این فرایند تا زمانی ادامه پیدا می‌کند که تعداد positive های باقی‌مانده از آستانه نویز کمتر شود یا دیگر rule مفیدی پیدا نشود. در نهایت مجموعه clause ها به عنوان مدل DNF برگردانده می‌شود

۱. اضافه شدن پارامتر epsilon

```
def learn_DNF_noisy(X, y, epsilon):
```

۲. تغییر معیار توقف الگوریتم

```
    np_thresh = epsilon * len(P_idx)
    while len(P_idx) > np_thresh:
```

الگوریتم زمانی متوقف می‌شود که تعداد positive های باقی‌مانده کمتر از حد نویز باشد.

۳. تغییر نحوه مدیریت شکست

```
    if not np.any(r):
        break
```

یعنی اگر rule پیشرفتی ایجاد نکند:

- الگوریتم متوقف می‌شود
- ولی مدل فعلی حفظ می‌شود.

۴. حذف شرط تناقض

در نسخه ی ساده، اگر rule هیچ positive ی را پوشش ندهد، شکست اتفاق افتاده است.

```
if len(covered_in_P) == 0: return None
```

در نسخه نویزی این شرط حذف شده است. زیرا:

- در داده noisy ممکن است rule کامل پیدا نشود.

تایع Predict using the Learned DNF

مانند تایع Predict_DNF نسخه قبلیست . تنها تفاوت در مدیریت rule خالی است.

```
satisfied = np.all(X[:, r] == 1, axis=1) if np.any(r) else  
np.zeros(X.shape[0], dtype=bool)
```

r در این کد یک آرایه بولی است که مشخص می کند چه featureهایی داخل rule انتخاب شده اند (True هستند)

np.any(r) بررسی می کند آیا حداقل یکی از عناصر r برابر True هست یا نه؟

در یادگیری نویزی ممکن است الگوریتم نتواند feature مناسبی پیدا کند. در این حالت rule ممکن است کاملاً خالی بماند:

r = [False, False, False, False]

یعنی:

- هیچ feature ای انتخاب نشده
- rule عملاً بی معنی است

بنابراین باید خالی بودن rule بررسی شود

تایع Turn the DNF model into a readable string

در تسک قبلی توضیح داده شد.

Main Experiment: Epsilon sweep 🚩

این بخش کد یک آزمایش برای پیدا کردن بهترین مقدار ϵ در مدل DNF نویزی انجام می‌دهد. ایده اصلی این است که بینیم با تغییر میزان خطای قابل قبول (ϵ) عملکرد مدل روی داده‌های اعتبارسنجی چگونه تغییر می‌کند.

```
X_train, y_train = x_noisy[:250], y_noisy[:250]
X_val, y_val = x_noisy[250:], y_noisy[250:]
```

ابتدا داده‌ها به دو بخش تقسیم می‌شوند:

- ۲۵۰ نمونه اول برای آموزش مدل استفاده می‌شوند.
- ۲۵۰ نمونه باقی‌مانده برای ارزیابی مدل استفاده می‌شوند.

هدف از validation این است که عملکرد مدل روی داده‌های دیده‌نشده بررسی شود.

```
epsilons = np.arange(0.001, 1.001, 0.05)
```

یعنی ϵ از حدود ۰ تا ۱ با گام ۰/۰۵ تغییر می‌کند. هر مقدار ϵ مشخص می‌کند الگوریتم چه مقدار خطا یا نویز را قابل قبول بداند.

- ϵ کوچک --> مدل سخت‌گیرتر
- ϵ بزرگ --> مدل انعطاف‌پذیرتر

```
val_errors = []
best_eps = None
best_err = np.inf
best_dnf = None
```

چند متغیر برای ذخیره بهترین نتیجه تعریف می‌شوند:

- `val_errors`: ذخیره خطای validation برای هر ϵ
- `best_err`: کمترین خطای مشاهده‌شده
- `best_eps`: بهترین مقدار ϵ
- `best_dnf`: بهترین مدل یادگرفته‌شده

`np.inf` یعنی بی‌نهایت، تا هر خطای واقعی از آن کوچک‌تر باشد.

```
for eps in epsilons:
```

برای هر مقدار ϵ حلقه اجرا می‌شود.

```
dnf = learn_DNF_noisy(X_train, y_train, eps)
```

مدل DNF نویزی با مقدار فعلی epsilon آموزش داده می شود.

```
y_pred = predict_DNF(dnf, X_val)
```

مدل روی داده های validation پیش بینی انجام می دهد.

```
err = np.mean(y_pred != y_val)
val_errors.append(err)
```

نرخ خطا محاسبه میشود. برای هر نمونه بررسی می کند آیا پیش بینی اشتباه بوده یا نه:

اگر حاصل $y_pred \neq y_val$

- True: (پیش بینی اشتباه)
- False: (پیش بینی درست)

در نهایت از پیش بینی های اشتباه میانگین گرفته میشود.

```
if err < best_err:
    best_err = err
    best_eps = eps
    best_dnf = dnf
```

اگر خطای فعلی بهتر از بهترین خطای قبلی باشد، بهترین مقادیر به روزرسانی می شوند. در پایان حلقه

- $best_eps <--$ بهترین مقدار epsilon
- $best_err <--$ کمترین خطا
- $best_dnf <--$ بهترین مدل یاد گرفته شده

خواهند بود.

Plot Error vs Epsilon sweep

این بخش کد نمودار تغییرات خطای validation نسبت به مقادیر مختلف epsilon را رسم می کند.

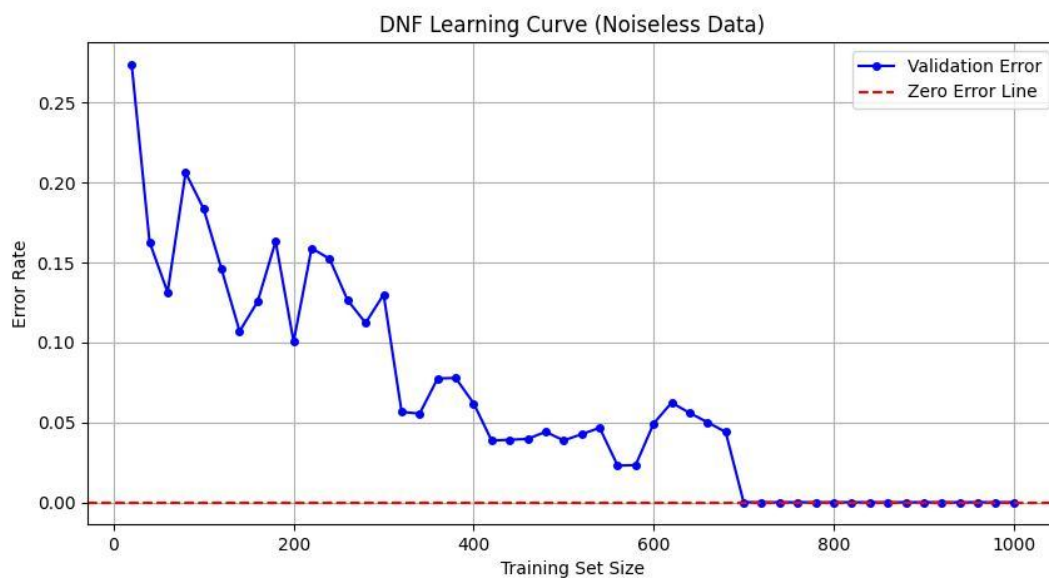
Task one Results:

Validation error first reached ZERO at training size = 700

Extracted DNF Formula:

Final DNF:

$$(x_1 \wedge x_2 \wedge x_3 \wedge x_4 \wedge x_5) \vee (x_{95} \wedge x_{96} \wedge x_{97} \wedge x_{98} \wedge x_{99} \wedge x_{100})$$



این خروجی نشان می‌دهد که مدل DNF روی داده بدون نویز عملکرد بسیار خوبی داشته است. با افزایش تعداد نمونه‌های آموزشی، نرخ خطا روی داده اعتبارسنجی به شکل یکنواخت کاهش یافته تا اینکه در اندازه ۷۰۰ نمونه برای اولین بار به صفر رسیده است. این یعنی مدل از آن نقطه به بعد توانسته است الگوی منطقی تولید داده را دقیقاً یاد بگیرد و دیگر هیچ پیش‌بینی اشتباهی انجام نداده است. روند نمودار نیز نشان می‌دهد که افزایش داده در ابتدا بیشترین تأثیر را داشته و نوسانات خطا با افزایش داده کمتر شده‌اند تا جایی که مدل پایدار و کاملاً دقیق شده است.

فرمول نهایی DNF نیز بسیار معنی‌دار است. مدل تنها دو قانون (clause) یاد گرفته است، که هر دو شامل تعداد زیادی feature هستند. اولین قانون از x_1 تا x_5 تشکیل شده و دومین قانون از x_{95} تا x_{100} این ساختار نشان می‌دهد که تولیدکننده داده در پس‌زمینه احتمالاً یک DNF ساده با دو بخش مجزا بوده است، و مدل دقیقاً همان دو بخش را بازسازی کرده است. این نتیجه کاملاً با رفتار یک مدل DNF در داده بدون نویز سازگار است: با افزایش کافی داده، الگوریتم Sequential Covering تمام الگوهای واقعی را درست استخراج کرده و به فرمول صحیح رسیده است.

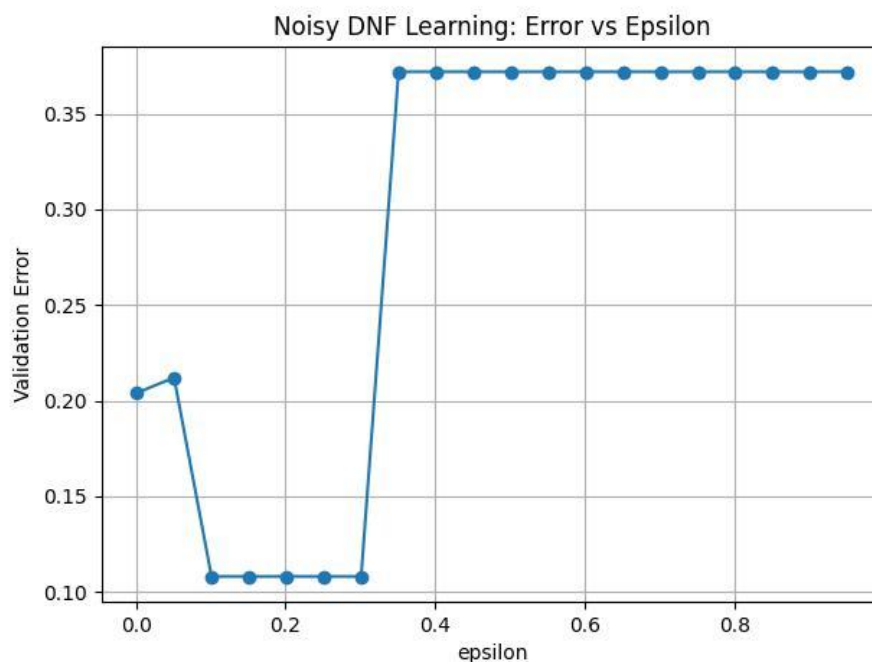
Task Two Results:

Best epsilon: 0.101

Minimum validation error: 0.108

DNF at best epsilon:

$(x_{30} \wedge x_{31}) \vee (x_{10} \wedge x_{11})$



این نمودار رفتار مدل DNF نویزی را هنگام تغییر مقدار epsilon نشان می‌دهد. در ابتدا، برای مقادیر خیلی کوچک epsilon مدل هنوز بیش از حد سخت گیر است و تلاش می‌کند نویز را هم حذف کند، بنابراین خطا نسبتاً بالا می‌ماند (حدود ۰.۲۰). اما وقتی epsilon کمی بزرگ‌تر می‌شود (نزدیک ۰.۱۰ تا ۰.۳۰)، مدل اجازه می‌دهد بخشی از نویز نادیده گرفته شود و الگوی اصلی داده بهتر استخراج شود. نتیجه این است که خطای validation ناگهان به محدوده‌ای بسیار پایین‌تر می‌رسد (حدود ۰.۱۰۸) که نشان‌دهنده تعادل مناسب میان ساده‌سازی و تحمل نویز است.

اما به محض اینکه مقدار epsilon خیلی بزرگ می‌شود (بیش از حدود ۰.۳۵)، مدل بیش از حد آزاد می‌شود. در این حالت ruleها اجازه دارند مقدار زیادی از داده‌های اشتباه یا نامرتبط را هم نادیده بگیرند، و همین باعث می‌شود مدل تقریباً معنای خاصی یاد نگیرد. به همین دلیل خطای validation ناگهان جهش کرده و در باقی محدوده epsilon تقریباً ثابت و بالا (حدود ۰.۳۷) باقی می‌ماند.

در مقدار بهینه $\epsilon = 0.101$ مدل توانسته بهترین عملکرد را ثبت کند. DNF یادگرفته‌شده نیز ساختاری نسبتاً ساده و قابل تفسیر دارد:

$$(x_{30} \wedge x_{31}) \vee (x_{10} \wedge x_{11})$$

این فرمول نشان می‌دهد که الگوریتم در حضور نویز موفق شده دو الگوی منطقی از داده استخراج کند که هر دو شامل دو ویژگی مجاور هستند. این سادگی و پایداری نشان می‌دهد که مدل در نقطه بهینه epsilon نویز را مدیریت کرده و الگوی واقعی داده را بدون پیچیدگی اضافه یاد گرفته است.

در این تمرین، الگوریتم DNF Learning در دو سناریوی بدون نویز و دارای نویز بررسی و ارزیابی شد. نتایج نشان می‌دهند که نسخه کلاسیک الگوریتم در محیط بدون نویز به خوبی عمل می‌کند و با افزایش اندازه داده‌های آموزشی به مقدار کافی، می‌تواند الگوی منطقی داده را دقیقاً بازیابی کند. همان‌طور که در منحنی یادگیری مشاهده شد، از حدود ۷۰۰ نمونه آموزشی به بعد، خطای اعتبارسنجی به صفر رسید و مدل توانست DNF واقعی داده را بدون هیچ انحرافی بازتولید کند.

اما در حالت نویزی رفتار کاملاً متفاوت است. داده شامل خطا باعث می‌شود نسخه ساده الگوریتم قادر به حذف کامل منفی‌ها نباشد، بنابراین نسخه تعدیل شده نویزی ارائه شد که در آن پارامتر ϵ کنترل می‌کند چه مقدار خطا مجاز است. آزمایش جستجوی ϵ نشان داد که مقدار مناسب حدود ۰.۱۰ است؛ جایی که مدل اجازه می‌دهد اندکی خطا نادیده گرفته شود اما هنوز از overfitting به نویز جلوگیری می‌شود. در این نقطه، مدل ساده‌ترین و پایدارترین فرمول منطقی را یاد گرفت:

$$(x_{30} \wedge x_{31}) \vee (x_{10} \wedge x_{11})$$

این دو قانون هم مختصرند و هم از نظر ساختار منطقی معنی‌دار به نظر می‌رسند، و بهترین نرخ خطای validation یعنی حدود ۰.۱۰۸ را تولید کرده‌اند.

در مجموع، آزمایش‌ها نشان می‌دهند که:

- در داده بدون نویز: الگوریتم DNF کلاسیک کاملاً موفق است و با داده کافی به عملکرد بی‌نقص می‌رسد.
- در داده نویزی: وجود پارامتر ϵ ضروری است؛ مقدار مناسب آن باعث می‌شود مدل الگوی واقعی را یاد بگیرد و در برابر نویز مقاوم باشد، در حالی که مقادیر خیلی بزرگ یا خیلی کوچک عملکرد را به شدت خراب می‌کنند.

این نتایج نقش حیاتی تنظیم نویز و اهمیت انتخاب صحیح hyperparameterها را در یادگیری مبتنی بر قواعد منطقی به خوبی نشان می‌دهند.